

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method `printArray` that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

```
1
2
3
Hello
World
```

Do not use method overloading because your answer will not be accepted.

23
24
25
26

```
printer.printArray(stringArray);
}
```

Line: 15 Col: 45

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Test case 0

Compiler Message

Success

Expected Output

11

22

33

4Hello

5World

Download

Problem

Submissions

Leaderboard

Discussions

Editorial

Write the following code in your editor below:

1. A class named `Arithmetic` with a method named `add` that takes **2** integers as parameters and returns an integer denoting their sum.
2. A class named `Adder` that inherits from a superclass named `Arithmetic`.

Your classes should not be **public**.

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the `add` method on an `Adder` object and passing it **2** integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your `add` method must return the sum of its parameters.

Sample Output

The main method in the `Solution` class above should print the following:

```
My superclass is: Arithmetic
42 13 20
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Line: 6 Col: 1

Test case 0

Compiler Message

Success

Expected Output

Download

```
1 My superclass is: Arithmetic
2 42 13 20
```

```

    }
    System.out.println("I am walking");
}

```

This class has only one method, walk. Next, we want to create a Bird class that also has a fly method. We do this using extends keyword:

```

class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
}

```

Finally, we can create a Bird object that can both fly and walk:

```

public class Solution{
    public static void main(String[] args){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
    }
}

```

The above code will print:

```

I am walking
I am flying

```

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique properties.

The code above is provided for you in your editor. You must add a sing method to the Bird class, then modify the main method accordingly so that the code prints the following lines:

```

I am walking
I am flying
I am singing

```

```

30     bird.sing();
31
32 }
33 }

```

Line: 36 Col: 44

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 6,00 points!

You are now 12 points away from the 4th star for your java badge.

83%

126/150

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

Test case 0

Compiler Message

Success

Expected Output

Download

```

1 I am walking
2 I am flying
3 I am singing

```

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerLand University has the following grading policy:

- Every student receives a *grade* in the inclusive range from 0 to 100.
- Any *grade* less than 40 is a failing grade.

Sam is a professor at the university and likes to round each student's *grade* according to these rules:

- If the difference between the *grade* and the next multiple of 5 is less than 3, round *grade* up to the next multiple of 5.
- If the value of *grade* is less than 38, no rounding occurs as the result will still be a failing grade.

Examples

- grade* = 84 round to 85 (85 - 84 is less than 3)
- grade* = 29 do not round (result is less than 38)
- grade* = 57 do not round (60 - 57 is 3 or higher)

Given the initial value of *grade* for each of Sam's *n* students, write code to automate the rounding process.

Function Description

Complete the function *gradingStudents* with the following parameter(s):

- int grades[n]*: the grades before rounding

Returns

- int[n]*: the grades after rounding

Input Format

The first line contains a single integer, *n*, the number of students.

Each line *i* of the *n* subsequent lines contains a single integer, *grades[i]*.

Constraints

- $1 \leq n \leq 60$
- $0 \leq grades[i] \leq 100$

Sample Input 0

```
4
73
67
38
33
```

Line: 51 Col: 1

Upload Code as FileTest against custom inputRun CodeSubmit Code

You have earned 10.00 points!
You are now 61.54 points away from the 4th star for your problem solving badge.

78%40346/4875

Congratulations

You solved this challenge. Would you like to challenge your friends?TwitterLinkedInNext Challenge

Test case 0Test case 1Test case 2Test case 3Test case 4Test case 5Test case 6

Compiler Message

Success

Input (stdin)

1 42 733 674 385 33

Expected Output

1 75

DownloadDownload

Problem

Submissions

Leaderboard

Discussions

Editorial

The Java instanceof operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an ArrayList with several instances of these classes. count method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

```
5
Student
Student
Rockstar
Student
Hacker
```

Sample Output

```
3 1 1
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 10.00 points!

You are now 17 points away from the 4th star for your java badge.

76%

133/150

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

Testcases

Test case 0

Test case 1

Test case 2

Java Class

Input (stdin)

Download

```
1 5
2 Student
3 Student
4 Rockstar
5 Student
6 Hacker
```

Expected Output

Download

```
1 3 1 1
```

Problem List

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

49 / 49 testcases passed

kimidibindu2 submitted at Sep 02, 2026 09:10

Analysis

Solution

1983.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

216 ms | Beats 93.19%

Memory

119.43 MB | Beats 56.98%



Code / Java

```
1 import java.util.*;
2
3 class ThroneInheritance {
4
5     class Person {
6         String name;
7         boolean alive;
8         List<Person> children;
9
10        Person(String name) {
11            this.name = name;
12            this.alive = true;
13            this.children = new ArrayList<>();
14        }
15
16        Person king;
17        Map<String, Person> people;
```

View more

More challenges

1993. Operations on Tree

Code

Java

Auto

1 import java.util.*;
2
3 class ThroneInheritance {
4
5 class Person {
6 String name;
7 boolean alive;
8 List<Person> children;
9
10 Person(String name) {
11 this.name = name;
12 this.alive = true;
13 this.children = new ArrayList<>();
14 }
15
16 Person king;
17 Map<String, Person> people;

Saved

In 33, Col 2

Testcase

Test Result

Accepted

Runtime: 4 ms

Case 1

Input

["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheritanceOrder","death","getInheritanceOrder"]
["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],["bob","alex"],["bob","asha"],[null],["bob"],[null]]

Output

[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha","catherine"],null,["king","andy","matthew","alex","asha","catherine"]]

Expected

[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha","catherine"],null,["king","andy","matthew","alex","asha","catherine"]]

Problem List

Accepted

Editorial

Solutions

Submissions

Accepted 39 / 39 testcases passed
vinidbendu2 submitted at Sep 02, 2025 09:17

1583.25ms
100.00%

Unlock the Full LeetCode Experience
Company problems, Ask Lekt, and expert editorials — all in one plan.

Runtime
12 ms | Beats 97.07%

Memory
54.21 MB | Beats 23.25%



Code / Java

```
1 class MyHashSet {
2     boolean[] set;
3
4     public MyHashSet() {
5         set = new boolean[1000001];
6     }
7
8     public void add(int key) {
9         set[key] = true;
10    }
11
12    public void remove(int key) {
13        set[key] = false;
14    }
15
16    public boolean contains(int key) {
17        return set[key];
18    }
19 }
```

More challenges
706. Design HashMap 1206. Design Skiplist

Code

Java

```
1 class MyHashSet {
2     boolean[] set;
3
4     public MyHashSet() {
5         set = new boolean[1000001];
6     }
7
8     public void add(int key) {
9         set[key] = true;
10    }
11
12    public void remove(int key) {
13        set[key] = false;
14    }
15
16    public boolean contains(int key) {
17        return set[key];
18    }
19 }
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["MyHashSet", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains"]
```

Output

```
[[], [1], [10001], [1], [1], [1], [7], [10007], [7], [7], [7], [123], [10123], [123], [123], [123], [5000], [15000], [5000], [5000], [5000]]
```

Expected

```
[null, null, null, null, null, false, null, null, null, null, false, null, null, null, null, false, null, null, null, null, false]
```

Problem List

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 73 / 73 testcases passed

kimidibindhu2 submitted at Sep 02, 2026 09:15

Analysis

Solution

Unlock the Full LeetCode Experience

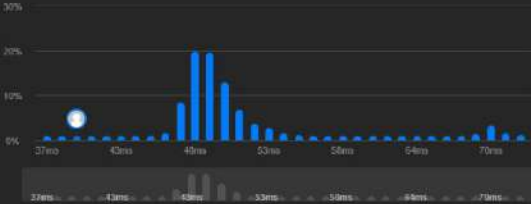
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

40 ms | Beats 99.45%

Memory

53.50 MB | Beats 44.95%



Code

Java

```
1 import java.util.*;
2
3 class BrowserHistory {
4     ArrayList<String> history;
5     int current;
6
7     public BrowserHistory(String homepage) {
8         history = new ArrayList<>();
```

More challenges

Code

Java

Auto

```
16         history.remove(history.size() - 1);
17     }
18
19     history.add(url);
20     current++;
21 }
22
23 public String back(int steps) {
24     current = Math.max(0, current - steps);
25     return history.get(current);
26 }
27
28 public String forward(int steps) {
29     current = Math.min(history.size() - 1, current + steps);
30     return history.get(current);
31 }
32 }
```

Testcase

Test Result

Accepted Runtime: 2 ms

Case 1

Input

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]

[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

Output

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Expected

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Problem List

Submit

Submit Ctrl Enter

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 57 / 57 testcases passed

Analysis

Solution

kinidibankhu2 submitted at Sep 02, 2025 09:13

7583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

111 ms | Beats 41.38%

Memory

57.62 MB | Beats 50.69%



Code

Java

```
1 import java.util.*;
2
3 class UndergroundSystem {
4
5     // Stores customer ID -> check-in station and time
6     Map<Integer, CheckIn> checkIns;
7
8     // Stores route -> total travel time and number of trips
9
10    }
11 }
```

More challenges

2166. Design Bitset

Java

Auto

```
35
36 public double getAverageTime(String startStation, String endStation) {
37     String route = startStation + "-" + endStation;
38
39     double[] data = routes.get(route);
40
41     return data[0] / data[1];
42 }
43
44 // Helper class
45 class CheckIn {
46     String station;
47     int time;
48
49     CheckIn(String station, int time) {
50         this.station = station;
51         this.time = time;
52     }
53 }
```

Saved

Testcase

Test Result

Accepted Runtime: 19 ms

Case 1

Case 2

Input

```
["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut","getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageTime"]
[[1],[45,"Leyton",31],[32,"Paradise",8],[27,"Leyton",10],[45,"Waterloo",15],[27,"Waterloo",20],[32,"Cambridge",22],[1,"Paradise","Cambridge"],[1,"Leyton","Waterloo"],[10,"Leyton",24],[1,"Leyton","Waterloo"],[10,"Waterloo",38],[1,"Leyton","Waterloo"]]
```

Output

```
[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]
```

Expected

```
[null,null,null,null,null,null,null,14.00000,11.00000,null,11.00000,null,12.00000]
```

Problem List

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 102 / 102 testcases passed

kimdbinhu2 submitted at Sep 02, 2024 09:03

AnalysisSolution

Unlock the Full LeetCode Experience

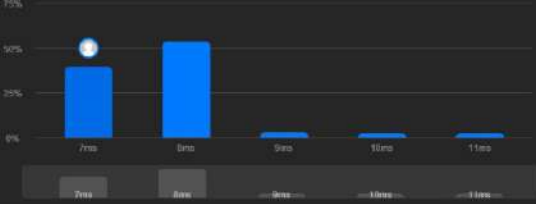
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

7 ms | Beats 100.00%

Memory

46.91 MB | Beats 64.92%



Category	Value
/ms	7ms
/ms	46.91 MB
/ms	
/ms	
/ms	

Code

Java

```
1 class ParkingSystem {
2     int[] slots;
3
4     public ParkingSystem(int big, int medium, int small) {
5         slots = new int[]{big, medium, small};
6     }
7
8     public boolean addCar(int carType) {
9         if (slots[carType] > 0) {
10             slots[carType]--;
11             return true;
12         }
13         return false;
14     }
15 }
```

More challenges

755. Pour Water

1894. Find the Student that Will Replace the Chalk

Code

Java

```
1 class ParkingSystem {
2     int[] slots;
3
4     public ParkingSystem(int big, int medium, int small) {
5         slots = new int[]{big, medium, small};
6     }
7
8     public boolean addCar(int carType) {
9         if (slots[carType] > 0) {
10             slots[carType]--;
11             return true;
12         }
13         return false;
14     }
15 }
```

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1

Input

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]

[[1, 1, 0], [1], [2], [3], [1]]

Output

[null, true, true, false, false]

Expected

[null, true, true, false, false]

Contribute a testcase